

 GAMERSTORE · DOCUMENTACIÓN TÉCNICA

Documentación · Backend

Spring Boot · qué hace cada archivo y qué modificar

Sistema de venta de tecnología · **Spring Boot** (API REST) + **React** (Vite) + **MySQL**

Los apartados marcados con  indican qué se puede modificar; los de , puntos delicados.

1. Cómo está organizado (las capas)

2. Configuración (lo que más vas a tocar)

- 2.1 Base de datos y arranque
- 2.2 Tienda y CORS
- 2.3 Seguridad JWT
- 2.4 Subida de imágenes
- 2.5 api-peru.dev (RENIEC)
- 2.6 Yape
- 2.7 Bloqueo de login
- 2.8 Stripe (pasarela real, modo prueba)
- 2.9 Comprobante y empresa emisora
- 2.10 El archivo application-local.properties

3. Seguridad (paquete config/security)

- 3.1 El flujo completo, paso a paso
- 3.2 Archivo por archivo
 - SecurityConfig.java
 - JwtService.java
 - JwtAuthenticationFilter.java
 - CustomUserDetailsService.java
 - RefreshTokenService.java
 - LoginAttemptService.java
 - LoginBloqueadoException.java

4. Modelo de datos (model/)

- 4.1 Diagrama de relaciones
- 4.2 Entidad por entidad
 - Producto → tabla producto
 - Categoría → tabla categoria
 - Cliente → tabla cliente
 - Pedido → tabla pedido
 - PedidoItem → tabla pedido_item
 - Pago → tabla pago
 - Comprobante → tabla comprobante
 - Usuario → tabla usuario
 - Rol (enum) → USUARIO, ADMIN
 - RefreshToken → tabla refresh_token

5. Repositorios (repository/)

- Cómo Spring Data genera las consultas
- ProductoRepository
- CategoriaRepository
- ClienteRepository
- PedidoRepository

PagoRepository
ComprobanteRepository
UsuarioRepository
RefreshTokenRepository

6. Servicios (service/) — el corazón

- 6.1 PagoService ★ (el más importante)
- 6.2 StripeService
- 6.3 CheckoutService
- 6.4 ComprobanteService
- 6.5 BoletaPdfService
- 6.6 ReniecService
- 6.7 PedidoService
- 6.8 ProductoService
- 6.9 CategoriaService
- 6.10 ClienteService
- 6.11 UsuarioService
- 6.12 PagoComprobanteService
- 6.13 PedidoReporteService

7. Controladores y endpoints (controller/)

- 7.1 Públicos (tienda, sin login)
- 7.2 Autenticación
- 7.3 Admin / ERP (todos exigen ROLE_ADMIN)
- 7.4 GlobalExceptionHandler

8. DTOs y Mappers

¿Por qué existen?

Familia: autenticación

Familia: producto y categoría

Familia: cliente

Familia: pedido

Familia: pago

Familia: comprobante y ventas

Familia: checkout

Familia: configuración y utilidades

Los 7 mappers

9. La boleta en PDF

- 9.1 Anatomía de templates/boleta.html
- 9.2 Cómo modificar el diseño ✎
- 9.3 NumeroALetras

10. Datos iniciales (DataSeeder)

Qué siembra

Cómo agregar o cambiar datos ✎

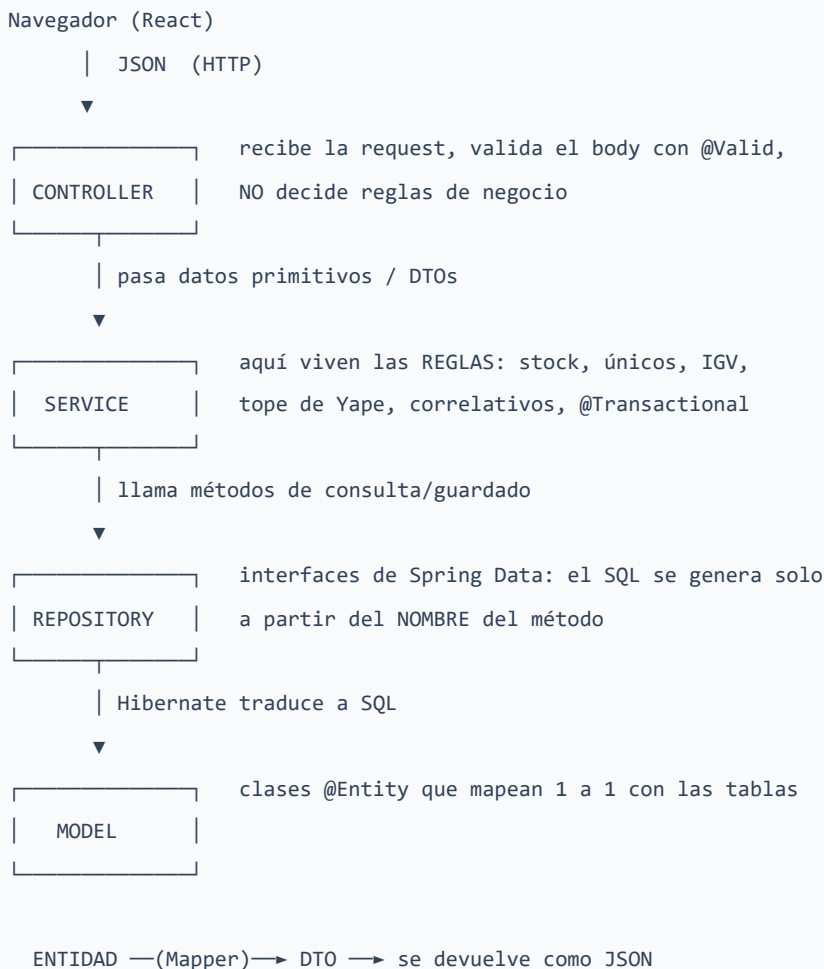
11. Guía rápida: "quiero cambiar X, ¿dónde toco?"

Proyecto: GamerStore — tienda gaming + panel administrativo (ERP) **Stack:** Spring Boot 3.5.6 · Java 17 · Spring Data JPA (Hibernate) · Spring Security + JWT (jjwt 0.12.6) · MariaDB/MySQL · Thymeleaf + openhtmltopdf · OpenPDF 1.3.35 · ZXing 3.5.3 · Stripe Java 29.2.0 **Paquete raíz:** `com.gamerstore.app` (`src/main/java/com/gamerstore/app`) **Punto de entrada:** `GamerStoreApplication.java` — `@SpringBootApplication` , arranca el contexto y ejecuta los `CommandLineRunner` (el `DataSeeder`).

Leyenda usada en todo el documento: ✏ = **modificable sin miedo** (es un valor de configuración o una decisión de negocio). ⚠ = **delicado** (tocarlo puede romper seguridad, integridad de datos o el flujo de compra).

1. Cómo está organizado (las capas)

El backend sigue el patrón clásico en capas. Una petición HTTP entra siempre por el mismo camino:



Capa	Carpeta	Responsabilidad	¿Qué NO debe hacer?
Controller	<code>controller/</code>	Definir rutas (<code>@GetMapping</code> , <code>@PostMapping</code> ...), validar el body con <code>@Valid</code> , convertir entidad→DTO con el mapper y devolver el JSON o el PDF.	No debe calcular precios, ni validar stock, ni tocar repositorios directamente.
Service	<code>service/</code>	Toda la lógica de negocio: validaciones de únicos, descuento de stock, cálculo de IGTV, correlativo de boleta, rechazo de pagos. Marca las transacciones con <code>@Transactional</code> .	No debe saber nada de HTTP más allá de lanzar <code>ResponseStatusException</code> con el código adecuado. No arma JSON ni HTML.
Repository	<code>repository/</code>	Consultas a la BD. Son interfaces que extienden <code>JpaRepository</code> ; Spring genera la implementación.	No debe contener reglas de negocio ni transformaciones de datos.
Model (entidad)	<code>model/</code>	Representar las tablas con <code>@Entity</code> , sus columnas y relaciones. Getters derivados sencillos (<code>getCodigo()</code> , <code>getSubtotal()</code>).	No debe llamar a repositorios ni servicios. No se devuelve nunca directamente al front.
DTO	<code>dto/</code>	Contrato de datos con el front: qué entra (<code>*Request</code>) y qué sale (<code>*DTO</code> , <code>*Response</code>). Son <code>record</code> inmutables con anotaciones de validación.	No debe tener lógica.
Mapper	<code>mapper/</code>	Convertir entidad → DTO en un solo lugar.	No debe consultar la BD.
Config	<code>config/</code> , <code>config/security/</code>	Beans de infraestructura: CORS, recursos estáticos, BCrypt, seguridad, JWT, semilla de datos.	No debe contener reglas de negocio de la tienda.
Util	<code>util/</code>	Helpers puros sin estado (<code>NumeroALetras</code>).	No debe depender de Spring.

Regla práctica para el equipo: si te preguntan "¿dónde pongo esta validación?" → en el **Service**. Si te preguntan "¿dónde cambio una URL?" → en el **Controller**. Si te preguntan "¿dónde cambio un valor?" → en **application.properties**.

2. Configuración (lo que más vas a tocar)

Archivo: `src/main/resources/application.properties`

2.1 Base de datos y arranque

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>spring.application.name</code>	Nombre lógico de la app en logs.	<code>gamerstore</code>	 Sí, cosmético.
<code>spring.profiles.active</code>	Perfil activo. Carga además <code>application-local.properties</code> .	<code>local</code>	 Si lo quitas, dejan de cargarse las claves privadas de Stripe/apiperu.
<code>server.port</code>	Puerto del backend.	<code>8080</code>	 Sí (si lo cambias, actualiza también la URL de la API en el front).
<code>spring.datasource.url</code>	Conexión JDBC. Incluye <code>createDatabaseIfNotExist=true</code> , así que XAMPP crea la BD solo.	<code>jdbc:mysql://127.0.0.1:3306/tienda_pc?...</code>	 Sí: cambia el host/puerto/nombre de BD según tu entorno.
<code>spring.datasource.username</code>	Usuario de MariaDB.	<code>root</code>	 Sí.
<code>spring.datasource.password</code>	Contraseña de MariaDB.	<code>(vacío)</code>	 Sí. En XAMPP por defecto va vacío.
<code>spring.jpa.hibernate.ddl-auto</code>	Hibernate crea/actualiza las tablas al arrancar leyendo las <code>@Entity</code> .	<code>update</code>	 No pongas <code>create</code> ni <code>create-drop</code> : borrarían todos los datos en cada arranque.
<code>spring.jpa.show-sql</code>	Imprime el SQL generado en consola.	<code>false</code>	 Ponlo en <code>true</code> para depurar y ver las consultas reales.

2.2 Tienda y CORS

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.tienda.nombre</code>	Nombre que muestra el front (lo entrega <code>GET /api/config</code>).	<code>GamerStore</code>	 Sí.
<code>app.whatsapp.numero</code>	Número de contacto que usa el front.	<code>51986969024</code>	 Sí.
<code>app.cors.allowed-origins</code>	Orígenes autorizados a llamar la API. Lo leen <code>WebConfig</code> y <code>SecurityConfig</code> (hacen <code>.split(",")</code> , así que admite varios separados por coma).	<code>http://localhost:5173</code>	 Sí.  En producción pon el dominio real, nunca <code>*</code> .

2.3 Seguridad JWT

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.jwt.secret</code>	Clave HMAC-SHA256 con la que se firman los access tokens (<code>JwtService</code>).	Cadena literal en el archivo	⚠ Sí, pero debe tener al menos 32 caracteres o <code>Keys.hmacShaKeyFor</code> falla. Cambiarla invalida todos los tokens ya emitidos. En producción debería ir en variable de entorno.
<code>app.jwt.access-expiration-ms</code>	Duración del access token.	<code>300000</code> (5 minutos)	✏ Sí, es el caso de cambio más habitual.
<code>app.jwt.refresh-expiration-ms</code>	Duración del refresh token guardado en BD.	<code>60480000</code> (7 días)	✏ Sí.

2.4 Subida de imágenes

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.uploads.dir</code>	Carpeta física donde <code>UploadController</code> guarda las imágenes; <code>WebConfig</code> la publica en <code>/images/productos/**</code> .	<code>uploads/products</code>	✏ Sí (ruta relativa al directorio de ejecución o absoluta).
<code>spring.servlet.multipart.max-file-size</code>	Peso máximo por archivo.	<code>5MB</code>	✏ Sí. Si lo cambias, actualiza también el mensaje de <code>GlobalExceptionHandler.tooLarge()</code> que dice "5MB".
<code>spring.servlet.multipart.max-request-size</code>	Peso máximo de la request completa.	<code>6MB</code>	✏ Sí; mantenlo por encima del anterior.

Nota: el QR de Yape **no** usa `app.uploads.dir`. `WebConfig` publica una segunda carpeta con ruta fija `uploads/qr` bajo la URL `/images/qr/**`.

2.5 apiperu.dev (RENIEC)

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.apidevperu.enabled</code>	Interruptor del autocompletado por DNI.	<code>true</code>	✏ Sí. En <code>false</code> , <code>ReniecService</code> ni siquiera llama a la API.
<code>app.apidevperu.base-url</code>	URL base del proveedor.	<code>https://apiperu.dev/api</code>	✏ Sí (si cambias de proveedor tendrás que ajustar también el mapeo del JSON).
<code>app.apidevperu.token</code>	Token Bearer. Se lee de la variable de entorno <code>APIDEVPERU_TOKEN</code> o del archivo local.	<code>\${APIDEVPERU_TOKEN:}</code>	⚠ Secreto. Va en <code>application-local.properties</code> , nunca en git.

2.6 Yape

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.yape.numero</code>	Número al que el comprador yapea. Se expone en <code>GET /api/config</code> y <code>GET /api/admin/pagos/config</code> .	912073109	 Sí.
<code>app.yape.titular</code>	Nombre del titular mostrado junto al QR.	Dietri Josue Diaz Asto	 Sí.
<code>app.yape.qr</code>	Ruta pública de la imagen del QR.	/images/qr/yape-qr.jpg	 Sí; el archivo físico va en <code>uploads/qr/</code> .
<code>app.yape.monto-maximo</code>	Tope de Yape: si el total del pedido lo supera, <code>PagoService.pagarConYape()</code> responde 409 y el front deshabilita esa opción.	500	 Sí, es una regla de negocio pura.

2.7 Bloqueo de login

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.login.max-intentos</code>	Intentos fallidos antes de bloquear (<code>LoginAttemptService</code>).	3	 Sí.
<code>app.login.bloqueo-segundos</code>	Duración del bloqueo temporal.	30	 Sí.

2.8 Stripe (pasarela real, modo prueba)

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.stripe.enabled</code>	Activa la pasarela real. Si está en <code>false</code> (o falta la clave secreta), el sistema cae automáticamente a la pasarela simulada con Luhn .	true	 Sí.
<code>app.stripe.public-key</code>	Clave pública; el navegador la usa para tokenizar la tarjeta. Se expone en <code>/api/config</code> .	<code>\${STRIPE_PUBLIC_KEY:}</code>	 Sí (no es secreta, pero va en el archivo local).
<code>app.stripe.secret-key</code>	Clave secreta que firma el cobro en <code>StripeService</code> .	<code>\${STRIPE_SECRET_KEY:}</code>	 Secreto absoluto. Nunca se expone en ningún endpoint.
<code>app.stripe.currency</code>	Moneda del <code>PaymentIntent</code> .	pen	 Sí, pero debe ser una moneda soportada por tu cuenta Stripe.

2.9 Comprobante y empresa emisora

Propiedad	Para qué sirve	Valor actual	¿Se puede cambiar?
<code>app.empresa.razon-social</code>	Nombre del emisor en la cabecera de la boleta.	GamerStore S.A.C.	 Sí.
<code>app.empresa.ruc</code>	RUC del emisor. Aparece dos veces en la boleta y es el primer campo del QR.	20601234567	 Sí (datos demo).
<code>app.empresa.direccion</code>	Dirección fiscal impresa.	Av. Javier Prado Este 4200, ...	 Sí.
<code>app.comprobante.serie</code>	Serie de la boleta. Define el prefijo del código (<code>B001-00000001</code>) y el correlativo se lleva por serie.	B001	 Sí. ⚠ Al cambiarla el correlativo arranca de nuevo en 1 para la serie nueva.
<code>app.comprobante.igv</code>	Tasa de IGV usada para desglosar el total.	0.18	 Sí. ⚠ Si la cambias, actualiza también el literal "I.G.V. (18%)" en <code>templates/boleta.html</code> , que está escrito a mano.

2.10 El archivo `application-local.properties`

- La **plantilla** versionada es `application-local.properties.ejemplo` . El archivo real (`application-local.properties`) está en `.gitignore` y **nunca se sube** — GitHub bloquea los push que contienen secretos.
- Contiene solo tres claves: `app.apidevperu.token` , `app.stripe.public-key` , `app.stripe.secret-key` .
- Cómo funciona el pisado de valores:** como `spring.profiles.active=local` , Spring carga primero `application.properties` y **encima** `application-local.properties` . Las propiedades repetidas en el archivo local **ganan**. Por eso `application.properties` puede declarar `app.stripe.secret-key=${STRIPE_SECRET_KEY:}` (vacío por defecto) y el archivo local lo rellena.
- Degradación elegante:** el sistema funciona igual sin claves. Sin token de apiperu, `ReniecService.consultarDni()` devuelve `Optional.empty()` y el DNI simplemente no se autocompleta. Sin clave de Stripe, `StripeService.estaActivo()` devuelve `false` y `CheckoutService` usa la pasarela simulada.

Para configurar tu entorno: copia `application-local.properties.ejemplo` → `application-local.properties` y rellena tus claves.

3. Seguridad (paquete `config/security`)

3.1 El flujo completo, paso a paso

A. Login (`POST /api/auth/login`)

1. `AuthController.login()` pregunta primero a `LoginAttemptService.segundosBloqueo(username)` . Si es > 0 lanza `LoginBloqueadoException` → **429**.
2. Llama a `authManager.authenticate(...)` . Spring Security usa `CustomUserDetailsService` para cargar el usuario (por username o por email) y compara la contraseña contra el hash BCrypt.
3. Si falla: `registrarFallo()` . Si ese fallo alcanza el máximo, se bloquea; si no, responde **401** indicando cuántos intentos quedan.
4. Si acierta: `limpiar()` borra el contador, `JwtService.generarAccess(u)` emite el access token (5 min) y `RefreshTokenService.crear(u)` guarda un refresh token nuevo en BD.

B. Cada request posterior

```
Request con "Authorization: Bearer eyJ..."
▼
JwtAuthenticationFilter (corre 1 vez por request)
  1. lee el header
  2. jwtService.extraerUsername(token)
  3. userDetailsService.loadUserByUsername(...)
  4. jwtService.esValido(token, username) → firma + expiración
  5. deja el Authentication en el SecurityContext
  ▼
SecurityConfig decide: ¿ruta pública / autenticada / ROLE_ADMIN?
  ▼
Controller
```

Si el token es inválido, el filtro **no lanza error**: simplemente no autentica, y `SecurityConfig` responde 401 o 403 según la ruta.

C. Refresh (`POST /api/auth/refresh`) — `RefreshTokenService.rotar()` valida que el token exista y esté vigente, lo marca `revocado = true` y crea uno nuevo (**rotación**: un refresh token solo sirve una vez). Devuelve access + refresh nuevos.

D. Logout (`POST /api/auth/logout`) — `revocar()` marca el token como revocado sin borrarlo.

3.2 Archivo por archivo

`SecurityConfig.java`

Configuración central. Es **el archivo que más se toca** cuando agregas endpoints.

- `csrf().disable()` — correcto aquí: la API es stateless y usa tokens, no cookies de sesión.
- `SessionCreationPolicy.STATELESS` — sin sesión de servidor.
- `authorizeHttpRequests` — ⚠ **las reglas se evalúan en orden**, la primera que coincide manda:

```

.requestMatchers("/api/auth/login", "/api/auth/refresh").permitAll()
.requestMatchers(HttpMethod.GET, "/api/productos/**", "/api/categorias/**", "/api/config/**").permitAll()
.requestMatchers("/images/**").permitAll()
.requestMatchers(HttpMethod.POST, "/api/checkout", "/api/checkout/**").permitAll()
.requestMatchers(HttpMethod.GET, "/api/checkout/boleto/**").permitAll()
.requestMatchers(HttpMethod.GET, "/api/reniec/**").permitAll()
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/**").authenticated()
.anyRequest().permitAll()           // ← la SPA de React

```

- `exceptionHandling` — devuelve JSON en vez del HTML de Spring: `{"error":"No autenticado"}` (401) y `{"error":"No tienes permisos"}` (403). ✏ Los mensajes son editables aquí mismo.
- `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)` — ⚠ el orden importa: el filtro JWT debe correr **antes** para que el `SecurityContext` ya esté poblado cuando se evalúan las reglas.
- `corsConfigurationSource()` — lee `app.cors.allowed-origins`. Expone `Content-Disposition` para que el navegador pueda leer el nombre de los PDF descargados.

✏ **Qué modificar aquí:** agregar rutas públicas, cambiar qué rutas exigen ADMIN, cambiar los mensajes de 401/403.

`JwtService.java`

Genera y valida los access tokens (HS256).

Método	Qué hace
<code>generarAccess(Usuario u)</code>	Firma un JWT con <code>subject = username</code> y claims <code>rol</code> , <code>nombre</code> , <code>id</code> .
<code>extraerUsername(String token)</code>	Devuelve el subject.
<code>esValido(String token, String username)</code>	Comprueba que el subject coincide y que no expiró. Devuelve <code>false</code> ante cualquier excepción.
<code>parse(String token)</code> (privado)	Verifica la firma; lanza excepción si el token fue alterado.

✏ **Modificable:** añadir claims al token (ej. el email) en `generarAccess`. ⚠ **Delicado:** la clave y el algoritmo.

`JwtAuthenticationFilter.java`

Extiende `OncePerRequestFilter`. Solo actúa si hay header `Authorization: Bearer`. Nótese la condición `SecurityContextHolder.getContext().getAuthentication() == null`: no pisa una autenticación ya establecida. El `catch (Exception ignored)` es intencional (token inválido = seguir sin autenticar).

`CustomUserDetailsService.java`

Puente entre Spring Security y la tabla `usuario`.

```

Usuario u = repo.findByUsername(loginId)
    .or(() -> repo.findByEmail(loginId))
    .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
return User.withUsername(u.getUsername())
    .password(u.getPassword())
    .authorities("ROLE_" + u.getRol().name())
    .build();

```

⚠ El prefijo `ROLE_` es obligatorio: `hasRole("ADMIN")` internamente busca la autoridad `ROLE_ADMIN`. Permite loguearse con **username o email**.

RefreshTokenService.java

Método	Qué hace
<code>crear(Usuario)</code>	Genera un token opaco (UUID sin guiones, no es un JWT) con su fecha de expiración.
<code>rotar(String token)</code>	Valida vigencia, revoca el actual y emite uno nuevo. 401 "Sesión inválida" / "Sesión expirada".
<code>revocar(String token)</code>	Marca <code>revocado = true</code> (logout).

✏ Modificable: la duración vía `app.jwt.refresh-expiration-ms`.

LoginAttemptService.java

Contador **en memoria** (`ConcurrentHashMap`), no usa BD. ⚠ Consecuencia a mencionar en la exposición: al reiniciar el servidor los contadores se pierden, y en un despliegue con varias instancias cada una llevaría su propia cuenta.

Método	Qué hace
<code>segundosBloqueo(username)</code>	Segundos que faltan; 0 si no está bloqueado. Si ya pasó, limpia el estado. Redondea hacia arriba con <code>Math.ceil</code> .
<code>registrarFallo(username)</code>	Suma un fallo; al llegar a <code>maxIntentos</code> activa el bloqueo y reinicia el contador.
<code>intentosRestantes(username)</code>	Para el mensaje "te quedan N intentos".
<code>limpiar(username)</code>	Se llama al loguear correctamente.

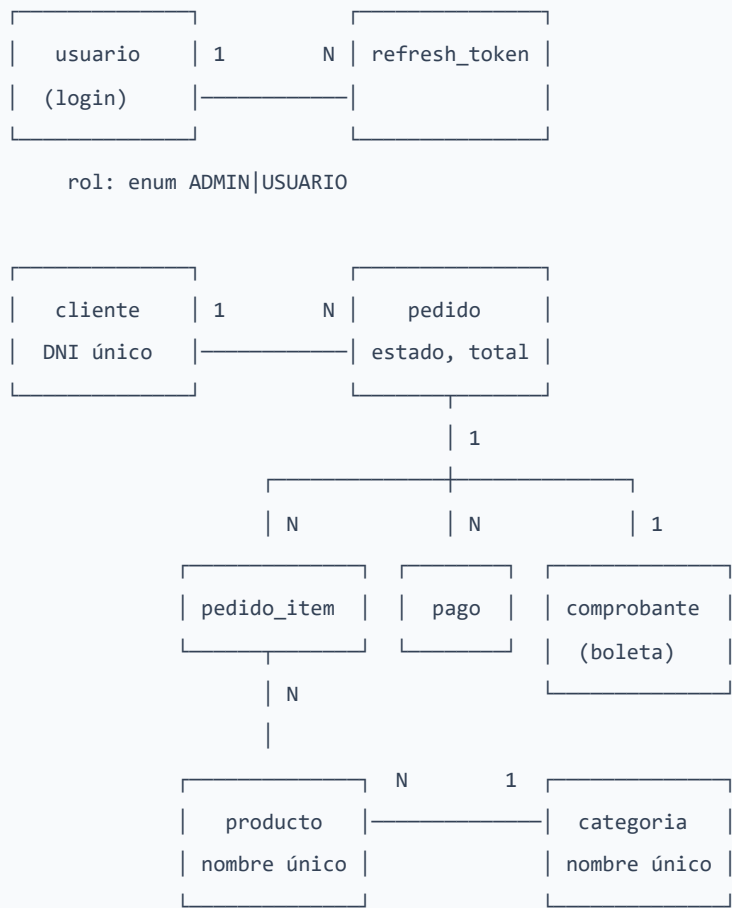
La clave del mapa se normaliza (`trim().toLowerCase()`), así que "Admin123" y "admin123 " cuentan como el mismo usuario.

LoginBloqueadoException.java

`RuntimeException` simple que lleva `segundosRestantes`. `GlobalExceptionHandler` la traduce a **429 Too Many Requests** incluyendo ese número, para que el front muestre una cuenta regresiva.

4. Modelo de datos (model/)

4.1 Diagrama de relaciones



Camino de una compra: cliente → pedido → (pedido_item × N) → pago → comprobante .

4.2 Entidad por entidad

Producto → **tabla** producto

Campo	Tipo / restricción
id	PK autoincremental
nombre	nullable=false, unique=true ⚠
descripcion	length=500
precio	double , obligatorio
imagen	length=500 — guarda la ruta (/images/productos/xxx.jpg), no el binario
stock	int
categoria	@ManyToOne(FetchType.EAGER) , FK categoria_id (nullable)

`Categoria` → **tabla** `categoria`

`id` + `nombre` (`nullable=false`, `unique=true`). Tiene constructor `Categoria(String nombre)` que usa el `DataSeeder` .

`Cliente` → **tabla** `cliente`

Campo	Detalle
<code>dni</code>	<code>unique=true</code> , <code>nullable=false</code> , <code>length=8</code> ⚠ es la clave natural del cliente
<code>nombres</code> , <code>apellidos</code>	obligatorios, 100
<code>telefono</code> (15), <code>email</code> (120), <code>direccion</code> (200)	opcionales
<code>fechaRegistro</code>	<code>@PrePersist onCreate()</code> la fija a <code>LocalDateTime.now()</code> si viene nula

Getter derivado: `getNombreCompleto()` → `nombres + " " + apellidos` . ⚠ Ojo: `email` **no** tiene `unique=true` a nivel de columna; la unicidad se valida en `ClienteService` y `CheckoutService` .

`Pedido` → **tabla** `pedido`

Campo	Detalle
<code>cliente</code>	<code>@ManyToOne</code> FK <code>cliente_id</code> , obligatorio
<code>fecha</code>	fijada por <code>@PrePersist</code>
<code>estado</code>	<code>length=30</code> , por defecto <code>"PENDIENTE"</code>
<code>total</code>	<code>double</code>
<code>metodoPago</code>	<code>length=30</code>
<code>items</code>	<code>@OneToMany(mappedBy="pedido", cascade=ALL, orphanRemoval=true, fetch=EAGER)</code>

⚠ El `cascade = ALL` es lo que permite guardar el pedido y sus líneas con un solo `save()` , y borrar las líneas al borrar el pedido.

Getters derivados: `getCodigo()` → `PED-0001` (`String.format("PED-%04d", id)`); `getCantidadTotal()` → suma de cantidades.

Estados usados en el código: `PENDIENTE` (al crear), `PAGADO` (al aprobarse el pago), `CANCELADO` (bloquea el cobro).

`PedidoItem` → **tabla** `pedido_item`

FK a `pedido` y a `producto` , más `cantidad` y `precioUnitario` . ⚠ Guarda el precio **al momento de la venta** (copiado de `producto.precio` en `PedidoService.crear`), así que cambiar el precio del catálogo no altera las ventas históricas. **Getter derivado:** `getSubtotal()` → `cantidad * precioUnitario` .

Pago → tabla pago

Campo	Detalle
pedido	@ManyToOne obligatorio
metodo	"YAPE" o "TARJETA"
monto , estado	"APROBADO" / "RECHAZADO"
referencia	obligatorio, 60 — N° de operación de Yape, código de autorización simulado, o el PaymentIntent id de Stripe
tarjetaUlt4 (4), titular (100), voucher (300)	opcionales
fecha	@PrePersist

Getter derivado: getCodigo() → PAG-0001 .

Comprobante → tabla comprobante

Campo	Detalle
tipo	por defecto "BOLETA" (solo se emiten boletas)
serie (6) + numero (int)	el correlativo se lleva por serie
pedido	@ManyToOne obligatorio
clienteNombre , clienteDni , clienteDireccion	⚠ snapshot: se copian al emitir. Si luego editas el cliente, la boleta emitida no cambia — que es exactamente lo que debe pasar con un comprobante.
subtotal , igv , total	desglose calculado
moneda	"PEN"
metodoPago , referenciaPago	copiados del pago
estado	"EMITIDO"
fechaEmision	@PrePersist

Getter derivado: getCodigo() → serie + "-" + String.format("%08d", numero) → B001-00000001 .

Usuario → tabla usuario

username (único, 50), email (único, 120), nombre , password (hash BCrypt), telefono , fechaRegistro , rol (@Enumerated(EnumType.STRING) , por defecto ADMIN). @PrePersist fija fecha y rol.

Rol (enum) → USUARIO , ADMIN

⚠ Se guarda como **texto** en la BD (EnumType.STRING). Si renombras un valor, los registros existentes dejan de mapear.

RefreshToken → **tabla** refresh_token

token (único, 100), usuario (FK), expiraEn (Instant), revocado (boolean). **Getter derivado:** estaVigente() →
!revocado && expiraEn.isAfter(Instant.now()) .

5. Repositorios (repository/)

Cómo Spring Data genera las consultas

No escribes SQL. Declaras un método en la interfaz y **Spring lee su nombre** para generar la consulta:

findBy	Categoría	Nombre	IgnoreCase	And	Nombre	Containing	IgnoreCase
verbo	propiedad	propiedad	modificador	unión	propiedad	LIKE %...%	
	(relación)	anidada					

Palabras clave que aparecen en este proyecto: `findBy`, `existsBy`, `countBy`, `And`, `IgnoreCase`, `Containing` (→ `LIKE %x%`), `LessThanEqual`, `IdNot` (→ `id <> ?`), `OrderBy...Asc/Desc`.

Cuando el nombre no alcanza, se usa `@Query` con JPQL (así están `ultimoNumero`, `sumTotal`, `topProductos` y `revocarTodosDe`).

ProductoRepository

Método	Consulta	Dónde se usa
<code>findByCategoriaNombreIgnoreCase(String)</code>	Productos de una categoría por nombre	<code>ProductoService.filtrar/porCategoría</code>
<code>findByNombreContainingIgnoreCase(String)</code>	Búsqueda por texto	<code>ProductoService.filtrar</code>
<code>findByCategoriaNombreIgnoreCaseAndNombreContainingIgnoreCase(...)</code>	Ambos filtros combinados	<code>ProductoService.filtrar</code>
<code>findByStockLessThanEqualOrderByStockAsc(int)</code>	Stock bajo, del más crítico al menos	<code>ProductoService.stockBajo</code> → dashboard
<code>existsByCategoriaId(Long)</code>	¿La categoría tiene productos?	<code>CategoriaService.eliminar</code> (bloquea el borrado)
<code>existsByNombreIgnoreCase(String)</code>	Nombre duplicado al crear	<code>ProductoService.crear/existeNombre</code>
<code>existsByNombreIgnoreCaseAndIdNot(String, Long)</code>	Nombre duplicado al editar	<code>ProductoService.actualizar/existeNombre</code>

CategoriaRepository

Método	Consulta	Dónde se usa
<code>findByNombre(String)</code>	Categoría exacta	disponible para uso general
<code>existsByNombreIgnoreCase(String)</code>	Duplicado al crear	<code>CategoriaService.crear</code>
<code>existsByNombreIgnoreCaseAndIdNot(String, Long)</code>	Duplicado al editar	<code>CategoriaService.actualizar</code>

ClienteRepository

Método	Consulta	Dónde se usa
<code>findByDni(String)</code>	Cliente por DNI	<code>CheckoutService.obtenerOCrearCliente</code> y <code>verificarCliente</code>
<code>existsByDni</code> / <code>existsByDniAndIdNot</code>	DNI único (alta / edición)	<code>ClienteService</code>
<code>existsByEmailIgnoreCase</code> / <code>existsByEmailIgnoreCaseAndIdNot</code>	Email único	<code>ClienteService</code> , <code>CheckoutService</code>
<code>findAllOrderByApellidosAscNombresAsc()</code>	Listado ordenado	<code>ClienteService.listar</code>

PedidoRepository

Método	Consulta	Dónde se usa
<code>findAllOrderByFechaDesc()</code>	Todos, más recientes primero	<code>PedidoService.todos</code>
<code>sumTotal()</code>	<code>SELECT COALESCE(SUM(p.total), 0) FROM Pedido p</code>	KPI de ventas del dashboard
<code>topProductos(Pageable)</code>	Agrupar <code>PedidoItem</code> por producto y suma cantidades, orden descendente. Devuelve <code>List<Object[]></code>	<code>AdminDashboardController</code> (top 5)

PagoRepository

Método	Consulta	Dónde se usa
<code>findAllOrderByFechaDesc()</code>	Listado de pagos	<code>PagoService.listar</code>
<code>existsByReferenciaAndMetodo(String, String)</code>	Evita registrar dos veces la misma operación Yape	<code>PagoService.pagarConYape</code>

ComprobanteRepository

Método	Consulta	Dónde se usa
<code>findAllOrderByFechaEmisionDesc()</code>	Registro de ventas	<code>ComprobanteService.listar</code>
<code>findByPedidoId(Long)</code>	Boleta de un pedido (a lo sumo una)	<code>emitir</code> (idempotencia) y <code>porPedido</code>
<code>ultimoNumero(String serie)</code>	<code>SELECT COALESCE(MAX(c.numero), 0) ... WHERE c.serie = :serie</code>	cálculo del correlativo

UsuarioRepository

Método	Consulta	Dónde se usa
<code>findByUsername</code> / <code>findByEmail</code>	Login con cualquiera de los dos	<code>CustomUserDetailsService</code> , <code>UsuarioService.buscar</code>
<code>existsByUsername</code> / <code>existsByEmail</code>	Únicos al crear	<code>UsuarioService.crear</code>
<code>existsByUsernameAndIdNot</code> / <code>existsByEmailAndIdNot</code>	Únicos al editar	<code>UsuarioService.actualizar</code>
<code>existsByRol(Rol)</code>	¿Hay algún admin?	<code>DataSeeder</code> (crear admin por defecto)
<code>countByRol(Rol)</code>	Cuántos admins hay	<code>UsuarioService.eliminar</code> (proteger el último)
<code>findAllOrderByUsernameAsc()</code>	Listado del panel	<code>UsuarioService.listar</code>

RefreshTokenRepository

Método	Consulta	Dónde se usa
<code>findByToken(String)</code>	Buscar el token	<code>RefreshTokenService.rotar/revocar</code>
<code>revocarTodosDe(Usuario)</code>	<code>@Modifying @Query</code> UPDATE que revoca todos los activos del usuario	disponible para invalidar todas las sesiones

6. Servicios (`service/`) — el corazón

6.1 `PagoService` ★ (el más importante)

Responsabilidad: cobrar un pedido y, si el cobro se aprueba, marcarlo como PAGADO y disparar la emisión de la boleta.

Método público	Qué hace
<code>listar()</code>	Todos los pagos, más reciente primero
<code>porId(Long)</code>	Un pago
<code>pagarConTarjeta(pedidoId, numero, titular, vencimiento, cvv)</code>	Pasarela simulada
<code>pagarConStripe(pedidoId, paymentMethodId)</code>	Pasarela real
<code>pagarConYape(pedidoId, numeroOperacion, voucher)</code>	Cobro por Yape

Reglas de negocio:

1. **Validación previa del pedido** (`validarPedido` , se aplica a los tres métodos): existe, no está `PAGADO` (409 "El pedido ya está pagado") ni `CANCELADO` (409 "El pedido está cancelado").
2. **Tarjeta simulada** — cuatro validaciones en cadena: - Formato: 13 a 19 dígitos tras quitar espacios y guiones. - **Algoritmo de Luhn** (`luhn()`): valida que el número sea matemáticamente consistente, igual que un banco real. - Vencimiento `MM/AA` , mes 1-12, y que el último día de ese mes no sea anterior a hoy. - CVV de 3 o 4 dígitos.
3. **Rechazo simulado** — ✏ esta es la regla que se enseña en la demo:

```
// Simulación del banco: los números que terminan en 0002 siempre se rechazan.  
boolean rechazado = limpio.endsWith("0002");
```

Cambiar el sufijo `"0002"` cambia qué tarjeta de prueba se rechaza.

4. **Tope de Yape** — ✏ configurable con `app.yape.monto-maximo` :

```
if (pedido.getTotal() > yapeMontoMaximo) {  
    throw new ResponseStatusException(HttpStatus.CONFLICT, String.format(  
        "Yape solo permite pagos de hasta S/ %, .2f. Usa tarjeta para montos mayores.", yapeMontoMaximo));  
}
```

5. **Anti-duplicado en Yape** — el número de operación debe tener 6 a 20 dígitos y no haber sido usado antes (`existsByReferenciaAndMetodo`) → 409 "Ese número de operación ya fue registrado".
6. **Aprobación** (`aprobarPedido` , privado) — el punto donde se enlazan pago, pedido y boleta:

```
private void aprobarPedido(Pedido pedido, Pago pago) {
    pedido.setEstado("PAGADO");
    pedido.setMetodoPago(pago.getMetodo());
    pedidoRepo.save(pedido);
    comprobanteService.emitir(pedido, pago); // ← la boleta nace aquí
}
```

7. **Código de autorización** — `generarCodigoAutorizacion()` produce 6 caracteres alfanuméricos con `SecureRandom`. Solo para la pasarela simulada; con Stripe la referencia es el id del `PaymentIntent`.

✂ **Qué se puede modificar:** el tope de Yape (properties), el sufijo de tarjeta rechazada, la longitud del código de autorización, el rango de dígitos del N° de operación, todos los mensajes de error. ⚠ **Delicado:** quitar `@Transactional` de los métodos de pago, o quitar la llamada a `comprobanteService.emitir()` (los pedidos quedarían pagados sin boleta).

6.2 StripeService

Responsabilidad: cobrar de verdad contra Stripe en modo prueba. El navegador tokeniza la tarjeta (los datos de tarjeta **nunca pasan por nuestro servidor**) y aquí solo llega un `paymentMethodId`.

Método	Qué hace
<code>estaActivo()</code>	<code>enabled && secretKey != null && !secretKey.isBlank()</code> — el interruptor que decide real vs. simulado
<code>cobrar(paymentMethodId, monto, descripcion)</code>	Devuelve el record <code>ResultadoStripe(aprobado, referencia, ult4, marca, mensaje, titular)</code>

Cómo funciona cobrar : 1. `PaymentMethod.retrieve(paymentMethodId)` → obtiene últimos 4 dígitos, marca y titular (los necesita el comprobante). 2. Crea y confirma un `PaymentIntent` con `.setConfirm(true)` y `AllowRedirects.NEVER` (sin 3-D Secure, la demo no maneja redirecciones). 3. ⚠ **Conversión de moneda:** `.setAmount(Math.round(monto * 100))` — Stripe trabaja en **céntimos**. 4. Si `status == "succeeded"` → aprobado.

Manejo de errores, dos casos distintos: - `CardException` (tarjeta rechazada por el banco) → **no es un error del sistema:** se loguea como `warn` y se devuelve `ResultadoStripe(false, ...)` - `StripeException` (no se pudo contactar la pasarela) → `log.error` + **502 Bad Gateway**.

✂ **Modificable:** la moneda (`app.stripe.currency`), la descripción del cargo, habilitar redirecciones si algún día se quiere 3-D Secure. ⚠ **Delicado:** `Stripe.apiKey = secretKey` se asigna en cada llamada (campo estático del SDK). Nunca expongas `secretKey` en un DTO.

6.3 CheckoutService

Responsabilidad: orquestar la compra pública completa. Es el único punto donde se juntan cliente + pedido + pago + boleta.

Método público	Qué hace
<code>comprar(CheckoutRequest)</code>	Todo el flujo. <code>@Transactional</code>
<code>verificarCliente(dni, email)</code>	Identifica a un comprador recurrente

Flujo de `comprar()` :

1. `obtenerOCrearCliente(req.cliente())` → busca por DNI; actualiza o crea
2. `pedidoService.crear(...)` → valida y descuenta stock, calcula total
3. `cobrar(pedido.getId(), req.pago())` → Yape / Stripe / simulado
4. si RECHAZADO → excepción → ROLLBACK TOTAL
5. busca el código de la boleta ya emitida
6. devuelve `CheckoutResponse`

Regla clave — el rollback transaccional. Es el mejor ejemplo de `@Transactional` para explicar en la exposición:

```
if ("RECHAZADO".equals(pago.getEstado())) {  
    throw new ResponseStatusException(HttpStatus.PAYMENT_REQUIRED,  
        "Pago rechazado por el banco. Revisa los datos de tu tarjeta e inténtalo de nuevo.");  
}
```

Al lanzar la excepción dentro de un método `@Transactional`, Spring revierte **toda** la transacción: no queda pedido creado, **el stock descontado se restaura** y el pago rechazado tampoco se guarda. El comprador puede reintentar sin dejar basura en la BD.

Regla — verificación de cliente recurrente. `verificarCliente` exige **DNI + email coincidentes**. Es una decisión de privacidad deliberada: con solo el DNI cualquiera podría recuperar los datos de un tercero. Si no coinciden → 404 "...Puedes continuar como invitado."

Regla — el email nunca rompe la compra. Tanto al crear como al actualizar el cliente, si el email ya pertenece a otro cliente simplemente **se omite** en lugar de fallar:

```
if (!clienteRepository.existsByEmailIgnoreCaseAndIdNot(c.email().trim(), cliente.getId())) {  
    cliente.setEmail(c.email());  
}
```

Regla — selección de pasarela (`cobrar`, privado): si Stripe está activo **y** llegó `paymentMethodId`, cobra real; si no, cae al simulado. Yape va por su rama. Cualquier otro método → 400.

 **Modificable:** los mensajes, qué campos del cliente se actualizan en compras sucesivas, agregar métodos de pago al `switch`.

6.4 `ComprobanteService`

Responsabilidad: emitir la boleta y alimentar el registro de ventas.

Método público	Qué hace
<code>emitir(Pedido, Pago)</code>	Crea la boleta. <code>@Transactional</code>
<code>listar()</code>	Todas, más reciente primero
<code>porId(Long)</code> / <code>porPedido(Long)</code> / <code>porPedidoCodigo(String)</code>	Búsquedas
<code>filtrar(desde, hasta)</code>	Filtro por rango de fechas, inclusive
<code>resumen(List<Comprobante>)</code>	Totales para el panel

Regla 1 — idempotencia. Si el pedido ya tiene boleta, la devuelve tal cual. Emitir dos veces es imposible.

Regla 2 — correlativo por serie:

```
int numero = comprobanteRepo.ultimoNumero(serie) + 1;
```

⚠ El propio código documenta la limitación: en producción esto necesitaría `SELECT ... FOR UPDATE` o una secuencia dedicada, porque dos pagos simultáneos podrían calcular el mismo número. Para una demo académica es aceptable — **menciónalo en la exposición, demuestra que entienden concurrencia.**

Regla 3 — el IGV va incluido en el precio (como en el retail peruano). No se suma, **se desglosa hacia atrás:**

```
double total = pedido.getTotal();
double subtotal = redondear(total / (1 + igvTasa));
double igv = redondear(total - subtotal);
```

Ejemplo: total S/ 118.00 → subtotal 100.00, IGV 18.00. `redondear()` usa `Math.round(valor * 100) / 100.0`.

Regla 4 — snapshot del cliente. Se copian nombre, DNI y dirección al comprobante: la boleta no cambia aunque después edites al cliente.

Nota de implementación: `porPedidoCodigo` recorre la lista en memoria filtrando por `getCodigo()` porque no existe índice por código de pedido. Aceptable con pocas boletas; ⚠ no escala.

✏ **Modificable:** serie e IGV vía properties; el tipo de comprobante (hoy siempre `"BOLETA"`) si algún día se agregara factura.

6.5 BoletaPdfService

Responsabilidad: convertir un `Comprobante` en un PDF, usando `templates/boleta.html` como plantilla.

Método público único: `generar(Comprobante c, List<PedidoItem> items) → byte[]`.

Cómo funciona: 1. Arma un `Context` de Thymeleaf con las variables `empresa` (mapa con `razonSocial`, `ruc`, `direccion`), `c` (el comprobante), `items`, `enLetras`, `qr` y `fechaTexto`. 2. `templateEngine.process("boleta", ctx) → HTML como String`. 3. `PdfRendererBuilder` de `openhtmltopdf` renderiza ese HTML+CSS a PDF (`useFastMode()`).

El QR — formato SUNAT (`textoQr`, privado):


```
return ruc + "|03|" + c.getSerie() + "|" + c.getNumero() + "|"
    + String.format("%.2f", c.getIgv()) + "|" + String.format("%.2f", c.getTotal()) + "|"
    + fecha + "|1|" + dni + "|";
```

Campos separados por `|`: RUC emisor · `03` (código SUNAT de boleta de venta) · serie · correlativo · IGV · total · fecha `yyyy-MM-dd` · `1` (tipo de documento del cliente = DNI) · número de documento.

Generación de la imagen (`generarQrDataUri`): ZXing `QRCodeWriter` produce un `BitMatrix` de 220×220 con margen 1, se pinta píxel a píxel en un `BufferedImage` y se codifica en Base64 como **data URI** (`data:image/png;base64,...`) que va directo en el `src` del `<img>`. Así el PDF no necesita ningún archivo externo.

⚠ El propio Javadoc advierte: la boleta **no se transmite a SUNAT** (haría falta RUC real y certificado digital), por eso no tiene validez tributaria y el PDF lleva un aviso rojo obligatorio.

6.6 ReniecService

Responsabilidad: autocompletar nombres y apellidos a partir del DNI consultando `apiperu.dev`.

Método público: `consultarDni(String dni) → Optional<ReniecPersona>`.

Filosofía "best-effort" — nunca rompe el flujo. Devuelve `Optional.empty()` (sin lanzar nada) si: el feature está apagado, falta el token, el DNI no tiene 8 dígitos, o **cualquier excepción de red/parseo** (se atrapa y se loguea como `warn`).

⚠ **Timeouts** — detalle importante, porque el `DataSeeder` llama a este servicio al arrancar:

```
factory.setConnectTimeout(2000);
factory.setReadTimeout(4000);
```

Sin esto, un `apiperu.dev` caído colgaría el arranque de la aplicación.

Mapeo del JSON: dos `record` privados. `ApiPeruData` usa `@JsonProperty` porque la API devuelve `snake_case` (`apellido_paterno`, `apellido_materno`, `nombre_completo`). Los apellidos se concatenan: `(paterno + " " + materno).trim()`.

✏ **Modificable:** los timeouts, el proveedor (`app.apidevperu.base-url` + ajustar los records), apagarlo con `enabled=false`.

6.7 PedidoService

Responsabilidad: alta de pedidos con cálculo de total y control de stock, más los datos para reportes.

Método público	Qué hace
<code>todos()</code> , <code>porId(Long)</code> , <code>total()</code>	Consultas básicas
<code>totalVentas()</code>	Suma de todos los totales (KPI)
<code>topProductos(int limite)</code>	Ranking con <code>PageRequest.of(0, limite)</code>
<code>reporte(desde, hasta, estado)</code>	Filtro en memoria para el PDF
<code>crear(clienteId, metodoPago, items)</code>	Alta completa. <code>@Transactional</code>
<code>actualizar(id, estado, metodoPago)</code>	Edición desde el panel
<code>eliminar(id)</code>	Borrado

Regla 1 — métodos de pago permitidos. Solo `TARJETA` o `YAPE` . El comentario del código explica por qué: `EFFECTIVO` / `PLIN` / `TRANSFERENCIA` eran del sistema anterior (venta cotizada por WhatsApp, sin pago ni boleta) y ya no aplican.

Regla 2 — validación y descuento de stock, la misma para el panel admin y el checkout público:

```
if (prod.getStock() < it.cantidad()) {
    throw new ResponseStatusException(HttpStatus.CONFLICT,
        "Stock insuficiente para " + prod.getNombre() + " (quedan " + prod.getStock() + ")");
}
PedidoItem item = new PedidoItem(pedido, prod, it.cantidad(), prod.getPrecio());
pedido.getItems().add(item);
total += item.getSubtotal();
prod.setStock(prod.getStock() - it.cantidad());
```

Regla 3 — el precio se congela. `prod.getPrecio()` se copia al `PedidoItem` ; el total se calcula sumando subtotales, nunca se confía en un total enviado por el cliente. ⚠️ Esto es también una medida de seguridad: el front no puede manipular el importe.

✏️ **Modificable:** los métodos de pago aceptados, los mensajes de stock.

6.8 ProductoService

Método	Regla
<code>todos()</code> , <code>porId()</code> , <code>porCategoria()</code> , <code>categorias()</code> , <code>total()</code>	Consultas
<code>filtrar(categoria, q)</code>	Elige el repo adecuado según qué filtros llegaron
<code>stockBajo(int umbral)</code>	Productos con <code>stock <= umbral</code>
<code>crear(...)</code>	⚠ Nombre único (409 "Ya existe un producto con ese nombre"). Fuerza <code>stock = Math.max(0, stock)</code>
<code>actualizar(...)</code>	Actualización parcial : solo aplica los campos no nulos / no vacíos. Valida nombre único excluyendo el propio id. <code>precio</code> solo si <code>> 0</code> , <code>stock</code> solo si <code>>= 0</code>
<code>ajustarStock(id, delta)</code>	Suma o resta; nunca deja negativo (<code>Math.max(0, ...)</code>)
<code>eliminar(id)</code>	Borrado directo
<code>existeNombre(nombre, id)</code>	Para la validación en vivo del formulario

6.9 CategoriaService

Método	Regla
<code>crear(nombre)</code>	Nombre obligatorio y único (<code>IllegalArgumentException</code> → 400)
<code>actualizar(id, nombre)</code>	Nombre único excluyendo el propio id
<code>eliminar(id)</code>	⚠ Bloqueo por integridad : <code>if (productoRepo.existsByCategoriaId(id)) throw ...</code> — "No se puede eliminar: la categoría tiene productos asociados"
<code>existeNombre(nombre, id)</code>	Validación en vivo

6.10 ClienteService

Método	Regla
<code>crear(...)</code>	DNI obligatorio; DNI único y email único → 409
<code>actualizar(...)</code>	Valida DNI y email únicos excluyendo el propio id. Nombres/apellidos solo se pisan si vienen no vacíos; teléfono/email/dirección se asignan siempre
<code>eliminar(id)</code>	Borrado. ⚠ Si el cliente tiene pedidos, la FK lo impide y <code>GlobalExceptionHandler</code> lo traduce a 409
<code>existeDni</code> / <code>existeEmail</code>	Validación en vivo

6.11 UsuarioService

Método	Regla
<code>autenticar(loginId, password)</code>	Busca por username, luego por email, y compara con <code>passwordEncoder.matches()</code>
<code>buscar(loginId)</code>	Username o email — se usa tras el login para armar el token
<code>crear(...)</code>	Contraseña obligatoria (400); username y email únicos (409). ⚠ Siempre guarda <code>passwordEncoder.encode(password)</code> , nunca texto plano
<code>actualizar(...)</code>	La contraseña solo se cambia si viene no vacía (así el formulario de edición puede dejarla en blanco)
<code>eliminar(id)</code>	⚠ Protección del último admin:
<code>parseRol(String)</code>	Convierte el texto a enum; si no es válido → 400 "Rol inválido"

```
if (u.getRol() == Rol.ADMIN && repo.countByRol(Rol.ADMIN) <= 1) {  
    throw new ResponseStatusException(HttpStatus.CONFLICT, "No puedes eliminar el último administrador");  
}
```

6.12 PagoComprobanteService

Genera con **OpenPDF** (programático, sin plantilla HTML) el comprobante de un pago: título, tabla clave/valor (fecha, cliente, pedido, método, referencia, tarjeta si aplica, estado) y el monto destacado a la derecha.

✏ **Modificable:** los colores `ACCENT` (`new Color(99, 102, 241)`) y `HEAD_BG` (`new Color(30, 27, 75)`), el formato de fecha `FMT`, y las filas que se agregan con `addFila(...)`. ⚠ No confundir con la **boleta** (`BoletaPdfService`): son dos PDF distintos. Este es el voucher del pago; aquel es el comprobante fiscal.

6.13 PedidoReporteService

Reporte PDF de pedidos, también con OpenPDF: título, línea de filtros aplicados, tabla de 7 columnas (Código, Cliente, Fecha, Estado, Método, Ítems, Total) y pie con total de ventas y cantidad de pedidos.

✏ **Modificable:** las columnas (array `cols` + los anchos en `new PdfPTable(new float[]{...})` — deben tener la misma longitud), los colores, el formato de fecha.

7. Controladores y endpoints (controller/)

7.1 Públicos (tienda, sin login)

Método	Ruta	Acceso	Qué hace	Service
GET	/api/productos	Público	Catálogo, filtros opcionales ? categoria=&q=	ProductoService.filtrar
GET	/api/productos/{id}	Público	Detalle + hasta 4 relacionados de la misma categoría	ProductoService
GET	/api/categorias	Público	Categorías para los filtros	CategoriaService.listar
GET	/api/config	Público	Nombre de tienda, WhatsApp, datos Yape, clave pública de Stripe	lee properties
GET	/api/reniec/{dni}	Público	Autocompletar por DNI (404 si no hay dato)	ReniecService
POST	/api/checkout	Público	Compra completa: cliente + pedido + pago + boleta	CheckoutService.comprar
POST	/api/checkout/cliente	Público	Identificar comprador recurrente (DNI + email)	CheckoutService.verificarCliente
GET	/api/checkout/boleta/{pedidoCodigo}?dni=	Público verificado	Descarga la boleta en PDF	ComprobanteService + BoletaPdfService

⚠ La descarga de boleta es pública pero **no abierta**: el controller compara el DNI del query param con `c.getClienteDni()` y responde **403** si no coincide, para que nadie descargue la boleta de otro adivinando el código.

7.2 Autenticación

Método	Ruta	Acceso	Qué hace	Service
POST	/api/auth/login	Público	Valida credenciales, emite access + refresh. 429 si está bloqueado	AuthenticationManager, JwtService, RefreshTokenService, LoginAttemptService
POST	/api/auth/refresh	Público	Rota el refresh y emite access nuevo	RefreshTokenService.rotar
POST	/api/auth/logout	Autenticado	Revoca el refresh (204)	RefreshTokenService.revocar
GET	/api/auth/me	Autenticado	Datos del usuario del token	UsuarioService.buscar

7.3 Admin / ERP (todos exigen `ROLE_ADMIN`)

Productos — `AdminProductoController` | Método | Ruta | Qué hace | |---|---|---| | GET | `/api/admin/productos` | Lista todos (incluidos sin stock) | | POST | `/api/admin/productos` | Crea (valida `ProductoRequest`) | | PUT | `/api/admin/productos/{id}` | Actualiza | | PATCH | `/api/admin/productos/{id}/stock?delta=` | Suma o resta stock | | DELETE | `/api/admin/productos/{id}` | Elimina (204) | | GET | `/api/admin/productos/existe?nombre=&id=` | Validación en vivo de duplicado |

Categorías — `AdminCategoriaController` : GET, POST, PUT `/api/admin/{id}`, DELETE `/api/admin/{id}`, GET `/api/admin/existe?nombre=&id=` sobre `/api/admin/categorias`.

Clientes — `AdminClienteController` | Método | Ruta | Qué hace | |---|---|---| | GET / POST / PUT `/api/admin/{id}` / DELETE `/api/admin/{id}` | `/api/admin/clientes` | CRUD | | GET | `/api/admin/clientes/reniec/{dni}` | Autocompletar desde RENIEC | | GET | `/api/admin/clientes/existe?dni=&email=&id=` | Valida DNI y email |

Usuarios — `AdminUsuarioController` : GET, POST, PUT `/api/admin/{id}` (204), DELETE `/api/admin/{id}` (204), GET `/api/admin/existe?username=&email=&id=` sobre `/api/admin/usuarios`.

Pedidos — `AdminPedidoController` | Método | Ruta | Qué hace | |---|---|---| | GET | `/api/admin/pedidos` | Lista | | GET | `/api/admin/pedidos/{id}` | Detalle | | POST | `/api/admin/pedidos` | Crea (descuenta stock) | | PUT | `/api/admin/pedidos/{id}` | Cambia estado / método | | DELETE | `/api/admin/pedidos/{id}` | Elimina | | GET | `/api/admin/pedidos/reporte.pdf?desde=&hasta=&estado=` | Reporte PDF (fechas `yyyy-MM-dd`) |

Pagos — `AdminPagoController` | Método | Ruta | Qué hace | |---|---|---| | GET | `/api/admin/pagos` | Lista de pagos | | POST | `/api/admin/pagos/tarjeta` | Cobro con tarjeta (Stripe si está activo y llega token; si no, simulado) | | POST | `/api/admin/pagos/yape` | Cobro con Yape | | GET | `/api/admin/pagos/config` | Cuenta Yape + estado y clave pública de Stripe | | GET | `/api/admin/pagos/{id}/comprobante.pdf` | Voucher del pago en PDF |

Comprobantes — `AdminComprobanteController` | Método | Ruta | Qué hace | |---|---|---| | GET | `/api/admin/comprobantes?desde=&hasta=` | Registro de ventas | | GET | `/api/admin/comprobantes/resumen?desde=&hasta=` | Cantidad, subtotal, IGV, total | | GET | `/api/admin/comprobantes/{id}/pdf` | Boleta por id | | GET | `/api/admin/comprobantes/pedido/{pedidoId}/pdf` | Boleta por pedido |

Dashboard — `AdminDashboardController` : GET `/api/admin/dashboard` devuelve los KPIs (totales de productos, categorías, clientes, pedidos, ventas), el **top 5** de productos y los productos con stock bajo. 🛠 El umbral es una constante en el propio controller: `private static final int UMBRAL_STOCK_BAJO = 10;`

Uploads — `UploadController` : POST `/api/admin/uploads` (multipart, campo `file`). Valida que no esté vacío y que el content-type empiece por `image/`; guarda con nombre UUID y extensión según el tipo (`.png`, `.webp`, `.gif`, por defecto `.jpg`); devuelve `{"url": "/images/productos/<uuid>.<ext>"}`.

7.4 GlobalExceptionHandler

`@RestControllerAdvice` que convierte toda excepción en JSON `{"error": "..."}.`

Excepción	HTTP	Mensaje / comportamiento
<code>MethodArgumentNotValidException</code>	400	Junta los mensajes de <code>@Valid</code> separados por <code>". "</code> y añade el mapa <code>errores</code> campo→mensaje
<code>ResponseStatusException</code>	el que traiga	Reenvía <code>e.getReason()</code> — es la vía normal desde los services
<code>IllegalArgumentException</code>	400	El mensaje de la excepción, o "Solicitud inválida"
<code>NoSuchElementException</code>	404	"Recurso no encontrado" (viene de los <code>orElseThrow()</code> sin argumento)
<code>AuthenticationException</code>	401	"Usuario o contraseña incorrectos"
<code>LoginBloqueadoException</code>	429	Mensaje + campo extra <code>segundosRestantes</code>
<code>DataIntegrityViolationException</code>	409	Red de seguridad: inspecciona el mensaje raíz. Si contiene <code>foreign key</code> / <code>referential integrity</code> / <code>fk_</code> / <code>constraint</code> → "No se puede eliminar: el registro está siendo usado por otros datos". Si contiene <code>unique</code> / <code>duplicate entry</code> / <code>already exists</code> → "Ya existe un registro con esos datos". Si no → "Operación no permitida: conflicto de datos"
<code>MaxUploadSizeExceededException</code>	413	"La imagen supera el tamaño máximo (5MB)"
<code>DateTimeParseException</code>	400	"Fecha inválida" (parámetros de reportes mal formados)

 **Todos estos mensajes son editables aquí.** Es el sitio correcto para cambiar la redacción de los errores genéricos; los específicos de negocio están en cada Service.

8. DTOs y Mappers

¿Por qué existen?

1. **No exponer las entidades.** Si devolvieras `Usuario` directamente, el JSON incluiría el **hash de la contraseña**. Con `UsuarioDTO` decides exactamente qué sale.
2. **Evitar bucles infinitos.** `Pedido` → `PedidoItem` → `Pedido` → ... Jackson entraría en recursión. Los mappers cortan el ciclo aplanando a `productoNombre`, `clienteNombre`, etc.
3. **Contrato estable.** Puedes renombrar una columna sin romper el front.
4. **Validación en la frontera.** Los `*Request` llevan `@NotBlank`, `@Pattern`, `@Email`, `@Min`, `@Size` ... y `@Valid` en el controller las dispara antes de llegar al Service.
5. **Inmutabilidad.** Todos son `record` de Java 17: sin setters, sin estado mutable.

Convención de nombres: `*Request` = entra · `*DTO` / `*Response` = sale.

Familia: autenticación

DTO	Para qué se usa
<code>LoginRequest</code>	username + password (ambos <code>@NotBlank</code>)
<code>LoginResponse</code>	accessToken, refreshToken, username, nombre, rol
<code>RefreshRequest</code>	refreshToken (<code>@NotBlank</code>) — también lo usa logout
<code>TokenResponse</code>	par access + refresh tras el refresh
<code>AuthUser</code>	respuesta de <code>/api/auth/me</code>

Familia: producto y categoría

DTO	Para qué se usa
<code>ProductoDTO</code>	Producto aplanado (incluye <code>categoriaId</code> y <code>categoriaNombre</code>)
<code>ProductoDetallado</code> <code>TO</code>	Producto + lista de relacionados
<code>ProductoRequest</code>	Alta/edición: nombre ≤ 100 , descripción ≤ 500 , precio <code>@DecimalMin("0.01")</code> , stock <code>@Min(0)</code> , categoría <code>@NotNull</code>
<code>CategoriaDTO</code>	id + nombre
<code>CategoriaRequest</code>	nombre <code>@NotBlank</code> ≤ 60

Familia: cliente

DTO	Para qué se usa
ClienteDTO	Cliente completo + nombreCompleto + fechaRegistro
ClienteRequest	Alta/edición: DNI @Pattern("\\d{8}") , nombres y apellidos obligatorios, teléfono \\d{0,15} , email @Email
ReniecPersona	nombres, apellidos, nombreCompleto (respuesta de apiperu)

Familia: pedido

DTO	Para qué se usa
PedidoDTO	Pedido + codigo , clienteNombre , cantidadTotal y sus líneas
PedidoItemDTO	Línea con productoNombre y subtotal
PedidoRequest	clienteld @NotNull , metodoPago, items @NotEmpty @Valid
PedidoItemRequest	productold @NotNull , cantidad @Min(1)
PedidoUpdateRequest	estado @NotBlank + metodoPago opcional

Familia: pago

DTO	Para qué se usa
PagoDTO	Pago aplanado con pedidoCodigo y clienteNombre
PagoTarjetaRequest	pedidold obligatorio; numero/titular/vencimiento/cvv (simulado) y paymentMethodId (Stripe) todos opcionales: cada flujo usa un subconjunto, y la validación real la hace PagoService
PagoYapeRequest	pedidold + numeroOperacion obligatorios, voucher opcional
YapeConfigDTO	numero, titular, qr, montoMaximo, stripeEnabled, stripePublicKey

Familia: comprobante y ventas

DTO	Para qué se usa
ComprobanteDTO	Boleta completa con codigo y pedidoCodigo
ResumenVentasDTO	cantidad, subtotal, igv, total
DashboardDTO	KPIs + stockBajo + topProductos
TopProductoDTO	Fila del ranking de más vendidos

Familia: checkout

DTO	Para qué se usa
<code>CheckoutRequest</code>	cliente + items + pago, los tres con <code>@Valid</code> anidado
<code>CheckoutClienteDTO</code>	Datos del comprador (mismas validaciones que <code>ClienteRequest</code>)
<code>CheckoutPagoDTO</code>	metodo <code>@NotBlank</code> + campos de tarjeta / Yape / Stripe
<code>CheckoutResponse</code>	Confirmación: códigos de pedido, pago y comprobante, estado, referencia, total, nombre
<code>VerificarClienteRequest</code>	DNI + email para identificar al comprador recurrente

Familia: configuración y utilidades

DTO	Para qué se usa
<code>ConfigDTO</code>	Config pública de la tienda
<code>ExisteDTO</code>	<code>{existe, mensaje}</code> de las validaciones en vivo
<code>UploadResponse</code>	URL de la imagen subida
<code>UsuarioDTO</code> / <code>UsuarioRequest</code>	Usuario del panel (el DTO nunca incluye password)

Los 7 mappers

Todos son `@Component` con un método `toDTO(...)` . Los interesantes:

Mapper	Qué aplana
<code>ProductoMapper</code>	<code>categoria</code> → <code>categoriaId</code> + <code>categoriaNombre</code> (con guarda de null)
<code>PedidoMapper</code>	Tiene además <code>toItemDTO()</code> ; convierte cada <code>PedidoItem</code> y resuelve <code>clienteNombre</code>
<code>PagoMapper</code>	Navega <code>pago</code> → <code>pedido</code> → <code>cliente</code> para sacar <code>pedidoCodigo</code> y <code>clienteNombre</code>
<code>ComprobanteMapper</code>	Resuelve <code>pedidoId</code> y <code>pedidoCodigo</code>
<code>ClienteMapper</code>	Añade el derivado <code>nombreCompleto</code>
<code>UsuarioMapper</code>	⚠ Omite deliberadamente el password
<code>CategoriaMapper</code>	id + nombre

⚠ Si agregas un campo a una entidad y quieres verlo en el front, tienes que tocar también el DTO y el mapper — la entidad sola no basta.

9. La boleta en PDF

Tres piezas trabajan juntas:

```
ComprobanteService.emitir() → crea el registro en BD (números, snapshot)
    ▼
BoletaPdfService.generar() → arma el Context + genera el QR (ZXing)
    ▼
templates/boleta.html → Thymeleaf rellena la plantilla
    ▼
openhtmltopdf → HTML + CSS → PDF (byte[])
    ▼
AdminComprobanteController / CheckoutController → descarga
```

9.1 Anatomía de `templates/boleta.html`

El archivo está dividido en **7 bloques comentados**, tanto en el `<style>` como en el `<body>`:

#	Bloque	Contenido
1	Cabecera	Razón social, dirección y RUC a la izquierda; recuadro con RUC, franja "BOLETA DE VENTA ELECTRÓNICA" y el código (<code>B001-00000001</code>) a la derecha
2	Tarjeta cliente	Señor(es), DNI, dirección · fecha de emisión, moneda, forma de pago y referencia
3	Tabla de ítems	CANT. (10%) · DESCRIPCIÓN (48%) · P. UNIT. (21%) · IMPORTE (21%), con filas alternadas
4	Totales	OP. GRAVADA, I.G.V. (18%), IMPORTE TOTAL (fila destacada)
5	Importe en letras	Recuadro punteado con el resultado de <code>NumeroALetras</code>
6	Pie	Texto legal + QR de 115×115 px
7	Aviso DEMO	Recuadro rojo: "DOCUMENTO DE DEMOSTRACIÓN — SIN VALIDEZ TRIBUTARIA"

9.2 Cómo modificar el diseño

Es **HTML y CSS normal**, todo dentro del mismo archivo. La paleta actual:

Color	Dónde se usa	Cómo cambiarlo
#4f46e5 (índigo)	Nombre de la empresa	<code>.empresa-nombre { color: ... }</code>
#1e1b4b (azul muy oscuro)	Borde y franja del recuadro, cabecera de la tabla, fila del total	Aparece en <code>.recuadro-boleta</code> , <code>table.items thead th</code> , <code>table.totales tr.fila-total</code>
#f8fafc (gris muy claro)	Fondo de la tarjeta del cliente y filas alternadas	<code>.tarjeta-cliente</code> , <code>tr.par</code>
#ef4444 / #fef2f2	Aviso DEMO	<code>.aviso-demo</code>

Recetas concretas: - **Cambiar el tamaño de hoja o márgenes** → `@page { size: A4; margin: 14mm; }` - **Cambiar los anchos de columna** → los `style="width:XX%"` de los `<th>` del bloque 3. - **Quitar el aviso DEMO** → borrar el `<div class="aviso-demo">`. ⚠ Recomendable mantenerlo mientras el proyecto sea académico. - **Agregar una columna a la tabla de ítems** → añade un `<th>` y un `<td th:text="...">`, y reajusta los porcentajes para que sumen 100. - **Cambiar el texto "I.G.V. (18%)"** → está **escrito a mano** en la línea del bloque 4. ⚠ Si cambias `app.comprobante.igv`, este literal no se actualiza solo. - **Agregar un dato nuevo** (ej. el teléfono de la empresa) → hay que hacerlo en **dos sitios**: añadirlo al `Map.of(...)` de `ctx.setVariable("empresa", ...)` en `BoletaPdfService` y luego usarlo con `th:text="${empresa.telefono}"` en la plantilla.

⚠ **Limitaciones de openhtmltopdf:** no es un navegador. Evita flexbox y grid; por eso el layout usa **tablas** (`table.encabezado`, `table.tarjeta-cols`, `table.pie`). Si metes CSS moderno puede que simplemente no se renderice.

9.3 NumeroALetras

Clase utilitaria `final` con constructor privado (no se instancia). Método público único:

```
NumeroALetras.convertir(double monto) → "SON: MIL NOVENTA Y OCHO CON 50/100 SOLES" .
```

Detalles del algoritmo que vale la pena conocer: - Redondea a centavos **antes** de separar para no arrastrar errores de coma flotante: `long centavosTotales = Math.round(Math.max(monto, 0) * 100);` - `UNIDADES[]` cubre 0–29 de corrido, porque en español los "teens" y los veinte-tantos son irregulares (ONCE, DIECISÉIS, VEINTIDÓS...). A partir de 30 se arma con `Y`: "TREINTA Y UNO". - Casos especiales del español resueltos: `100` → **CIEN** (no "CIENTO"), `1000` → **MIL** (no "UN MIL"), `1.000.000` → **UN MILLÓN**. - `apocopar()` convierte "VEINTIUNO" → "VEINTIUN" cuando antecede a MIL/MILLONES. El último grupo **no** se apocopa porque le sigue "CON". - Soporta hasta 999.999.999.

✏ Para cambiar la moneda (ej. a dólares) hay que editar el literal `"/100 SOLES"` en `convertir()`.

10. Datos iniciales (`DataSeeder`)

`@Component` que implementa `CommandLineRunner`: Spring Boot ejecuta `run()` automáticamente al arrancar.

Qué siembra

Bloque	Condición (idempotencia)	Contenido
Categorías	<code>categoriaRepo.count() == 0</code>	10: Tarjetas Graficas, Procesadores, Placas Madre, Memorias RAM, Almacenamiento, Monitores, Perifericos, Audio, Sillas Gamer, Consolas
Productos	<code>productoRepo.count() == 0</code>	~28 productos reales de tecnología con precios en soles, stock e imagen local
Clientes	<code>clienteRepo.count() == 0</code>	6 clientes con DNI, teléfono, email y dirección
Admin	<code>!usuarioRepo.existsByRol(Rol.ADMIN)</code>	<code>admin123</code> / <code>gamerstore123</code>
Pedidos	—	⚠ Ya no se siembran. El comentario del código lo explica: toda venta debe entrar por el checkout con pago real y boleta; sembrar ventas ficticias sin pago ni comprobante rompía esa coherencia

Idempotencia: cada bloque comprueba si su tabla ya tiene datos. Puedes reiniciar la app cuantas veces quieras sin duplicar nada. Si el bloque de categorías se salta, igual carga el mapa `cats` desde la BD para que los productos puedan enlazarse.

Integración con RENIEC: el helper `clienteReal(...)` consulta `apiperu.dev` por DNI y, si responde, usa los nombres reales; si no, usa los de respaldo pasados por parámetro. Por eso `ReniecService` tiene timeouts cortos: un servicio caído no puede colgar el arranque.

Cómo agregar o cambiar datos

Agregar un producto — añade una línea en el bloque de productos:

```
productoRepo.save(new Producto("Nombre del producto", "Descripción corta",
    1299.00, img("slug-imagen"), 12, cats.get("Perifericos")));
```

El helper `img("slug")` construye `/images/productos/slug.jpg`, así que coloca el archivo `slug.jpg` en `uploads/productos/`. El último parámetro debe ser una clave **exacta** del mapa `cats` (⚠ los nombres van sin tildes: `"Tarjetas Graficas"`, `"Perifericos"`).

Agregar una categoría — añádela al array `nombres` del bloque de categorías.

⚠ **Para ver los cambios**, la tabla debe estar vacía: como el seeder solo actúa con `count() == 0`, tendrás que vaciar `producto` (o `categoria`) desde phpMyAdmin antes de reiniciar. Recuerda respetar el orden de las FK: primero `pedido_item` / `comprobante` / `pago`, luego `pedido`, luego `producto`.

Cambiar las credenciales del admin — edita el bloque final. ⚠ Solo se crea si **no existe ningún** usuario con rol ADMIN; si ya tienes uno, cambia la contraseña desde el panel.

11. Guía rápida: "quiero cambiar X, ¿dónde toco?"

#	Quiero...	Archivo / propiedad	Detalle
1	Cambiar el IGV	<code>application.properties</code> → <code>app.comprobante.igv</code>	Ej. <code>0.18</code> → <code>0.10</code> . Lo lee <code>ComprobanteService</code> , que desglosa <code>subtotal = total / (1 + igv)</code> . ⚠ Actualiza también el literal "I.G.V. (18%)" en <code>templates/boleta.html</code> , que está escrito a mano.
2	Cambiar la serie de boleta	<code>app.comprobante.serie</code>	✏ <code>B001</code> → <code>B002</code> . Cambia el prefijo del código (<code>B002-00000001</code>). ⚠ El correlativo se lleva por serie , así que la nueva arranca en 1.
3	Cambiar los datos de la empresa	<code>app.empresa.razon-social</code> , <code>app.empresa.ruc</code> , <code>app.empresa.direccion</code>	✏ Los lee <code>BoletaPdfService</code> y los pasa a la plantilla. El RUC además es el primer campo del QR .
4	Cambiar el tope de Yape	<code>app.yape.monto-maximo</code>	✏ Un solo valor. Backend: <code>PagoService.pagarConYape</code> responde 409. Front: llega por <code>/api/config</code> y deshabilita la opción.
5	Cambiar el número o el QR de Yape	<code>app.yape.numero</code> , <code>app.yape.titular</code> , <code>app.yape.qr</code> + archivo en <code>uploads/qr/</code>	✏ La imagen física va en <code>uploads/qr/</code> (ruta fija en <code>WebConfig</code>), publicada bajo <code>/images/qr/**</code> .
6	Cambiar intentos de login o bloqueo	<code>app.login.max-intentos</code> , <code>app.login.bloqueo-segundos</code>	✏ Los lee <code>LoginAttemptService</code> con <code>@Value</code> . Contador en memoria : se pierde al reiniciar.
7	Cambiar la duración del token	<code>app.jwt.access-expiration-ms</code> (y <code>refresh-expiration-ms</code>)	✏ En milisegundos : 5 min = <code>300000</code> , 1 h = <code>3600000</code> . Lo lee <code>JwtService</code> por constructor.
8	Agregar un producto o categoría al seeder	<code>config/DataSeeder.java</code>	✏ Nueva línea <code>productoRepo.save(new Producto(...))</code> o entrada en el array <code>nombres</code> . ⚠ Solo corre si la tabla está vacía (<code>count() == 0</code>).
9	Hacer pública una ruta nueva	<code>config/security/SecurityConfig.java</code> → <code>authorizeHttpRequests</code>	✏ Añade <code>.requestMatchers(HttpMethod.GET, "/api/mi-ruta/**").permitAll()</code> . ⚠ Antes de <code>.requestMatchers("/api/**").authenticated()</code> — las reglas se evalúan en orden.
10	Cambiar el diseño de la boleta	<code>src/main/resources/templates/boleta.html</code>	✏ HTML + CSS en un solo archivo, 7 bloques comentados. Colores en el <code><style></code> . ⚠ Sin flexbox ni grid: openhtmltopdf no los soporta bien, por eso el layout usa tablas.

#	Quiero...	Archivo / propiedad	Detalle
1	Agregar un campo a un producto	5 archivos en cadena	<code>model/Producto.java</code> (campo + <code>@Column</code> + getter/setter) → <code>dto/ProductoDTO.java</code> → <code>dto/ProductoRequest.java</code> (con su validación) → <code>mapper/ProductoMapper.java</code> → <code>service/ProductoService.java</code> (crear y actualizar) → <code>controller/AdminProductoController.java</code> (pasar el nuevo parámetro) → y el formulario del front. ⚠ Con <code>ddl-auto=update</code> Hibernate agrega la columna solo.
1			
1	Cambiar el umbral de stock bajo	<code>controller/AdminDashboardController.java</code>	✏ <code>private static final int UMBRAL_STOCK_BAJO = 10;</code> — está hardcodedo, no es una property.
2			
1	Pasar Stripe a producción	<code>application-local.properties</code>	⚠ Reemplaza <code>pk_test_...</code> / <code>sk_test_...</code> por las claves live del dashboard de Stripe. ⚠ Con claves live los cobros son dinero real . Nunca subas la clave secreta a git.
3			
1	Cambiar el tamaño máximo de imagen	<code>spring.servlet.multipart.max-file-size</code> y <code>max-request-size</code>	✏ Ej. <code>5MB</code> → <code>10MB</code> (sube también <code>max-request-size</code>). ⚠ Actualiza el mensaje "(5MB)" en <code>GlobalExceptionHandler.toolarge()</code> .
4			
1	Cambiar los tipos de imagen aceptados	<code>controller/UploadController.java</code>	✏ El <code>switch (ct)</code> que mapea content-type → extensión, y la validación <code>ct.startsWith("image/").</code>
5			
1	Cambiar un mensaje de error	El Service correspondiente, o <code>GlobalExceptionHandler</code>	✏ Mensajes de negocio → dentro del <code>ResponseStatusException</code> del Service (ej. "Stock insuficiente para..." en <code>PedidoService.crear</code>). Mensajes genéricos (404, 409, 413) → <code>GlobalExceptionHandler</code> . Mensajes de 401/403 de seguridad → <code>SecurityConfig</code> .
6			
1	Cambiar qué tarjeta se rechaza en la demo	<code>service/PagoService.java</code>	✏ <code>boolean rechazado = limpio.endsWith("0002");</code>
7			
1	Cambiar el nombre de la tienda o el WhatsApp	<code>app.tienda.nombre</code> , <code>app.whatsapp.numero</code>	✏ Llegan al front por <code>GET /api/config</code> .
8			
1	Permitir otro origen (CORS)	<code>app.cors.allowed-origins</code>	✏ Admite varios separados por coma. Lo leen <code>WebConfig</code> y <code>SecurityConfig</code> . ⚠ Nunca uses <code>*</code> en producción.
9			
2	Cambiar los colores de los PDF de pago/reporte	<code>PagoComprobanteService.java</code> , <code>PedidoReporteService.java</code>	✏ Constantes <code>ACCENT = new Color(99, 102, 241)</code> y <code>HEAD_BG = new Color(30, 27, 75)</code> . Son PDF programáticos (OpenPDF), no usan plantilla HTML.
0			
2	Cambiar las columnas del reporte de pedidos	<code>service/PedidoReporteService.java</code>	✏ El array <code>cols</code> y los anchos de <code>new PdfPTable(new float[]{...})</code> . ⚠ Ambos deben tener la misma longitud.
1			

#	Quiero...	Archivo / propiedad	Detalle
2	Cambiar los métodos de pago aceptados	<code>service/PedidoService.crear + CheckoutService.comprar</code>	✎ La validación que hoy solo admite <code>TARJETA</code> / <code>YAPE</code> , y el <code>switch</code> de <code>CheckoutService</code> .
2			
2	Desactivar el autocompletado por DNI	<code>app.apidevperu.enabled=false</code>	✎ <code>ReniecService</code> deja de llamar a la API y devuelve <code>Optional.empty()</code> ; nada más se rompe.
3			
2	Ver el SQL que genera Hibernate	<code>spring.jpa.show-sql=true</code>	✎ Muy útil para depurar y para explicar en la exposición qué consulta genera cada método del repositorio.
4			

Resumen para la exposición

Si tienes que quedarte con cinco ideas del backend:

1. **Capas estrictas.** El controller no calcula nada; el service tiene todas las reglas; el repositorio no sabe de negocio. Ninguna entidad sale al front sin pasar por un mapper.
2. **Seguridad stateless.** No hay sesión de servidor: cada request se autentica con su JWT. El refresh token sí vive en BD y **rota** (un solo uso), y hay bloqueo por intentos fallidos.
3. **Transaccionalidad real.** Si el banco rechaza el pago, `@Transactional` revierte pedido, stock y pago de una sola vez. No quedan datos a medias.
4. **El IGV va incluido y se desglosa hacia atrás** (`total / 1.18`), como en el retail peruano — no se suma al final.
5. **Degradación elegante.** Sin token de RENIEC no se autocompleta pero se compra igual; sin claves de Stripe la pasarela cae a la simulada con Luhn. La demo nunca se cae por una dependencia externa.